HANDS-ON 3 [Beginner]
Django REST Views, URL Routing & Forms
Topics Covered:
Function-Based Views (FBV)✓
Class-Based Views (CBV)✓
URL Routing with include()✓
Django REST Framework (DRF) basics✓
Serializers✓
Request & Response objects✓
You will now build the actual REST API endpoints for the Course Management system using Django REST Framework — the most widely used package for building APIs in Django.
INSTALL
pip install djangorestframework then add 'rest_framework' to INSTALLED_APPS in settings.py
Task 1: Serializers and Basic API Views
Goal: Create DRF serializers and your first API views.
Steps:
26.
In courses/serializers.py, create a ModelSerializer for each model: DepartmentSerializer, CourseSerializer, StudentSerializer, EnrollmentSerializer. Include all fields.
27.
In courses/views.py, create a CourseListView using DRF's APIView: handle GET (return all courses serialized) and POST (create a new course from request.data).
28.
Create a CourseDetailView for GET (single course by pk), PUT (update), and DELETE operations.
29.
Wire both views in courses/urls.py: path('courses/', CourseListView.as_view()) and path('courses/<int:pk>/', CourseDetailView.as_view()). Include courses/urls.py in the main urls.py.
30.
Test all endpoints using Postman or Thunder Client: GET /api/courses/ should return a JSON list; POST with a JSON body should create a course; DELETE /api/courses/1/ should remove it.
Hint:
•
ModelSerializer auto-generates fields from the model — you only need to override for custom validation or computed fields.
•
Return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST) when the serializer is invalid — never silently ignore validation errors.

Expected Outcome: All 5 HTTP methods (GET list, GET detail, POST, PUT, DELETE) work correctly and return appropriate status codes (200, 201, 204, 400, 404).

Task 2: ViewSets and Routers

Goal: Refactor views to use DRF ViewSets and automatic URL routing.

Steps:

31.
Replace CourseListView and CourseDetailView with a single CourseViewSet that extends viewsets.ModelViewSet. This gives you all 5 CRUD operations with just 3 lines of code.

32.
Create a DefaultRouter in courses/urls.py and register the viewset: router.register('courses', CourseViewSet). Include router.urls. Observe how the router auto-generates all URL patterns.

33.
Do the same for StudentViewSet and EnrollmentViewSet.

34.
Add a custom action to CourseViewSet using the @action decorator: a GET endpoint /api/courses/{id}/students/ that returns all students enrolled in that course.

35.
Test the custom action endpoint in Postman. Verify it returns only students enrolled in the specified course.

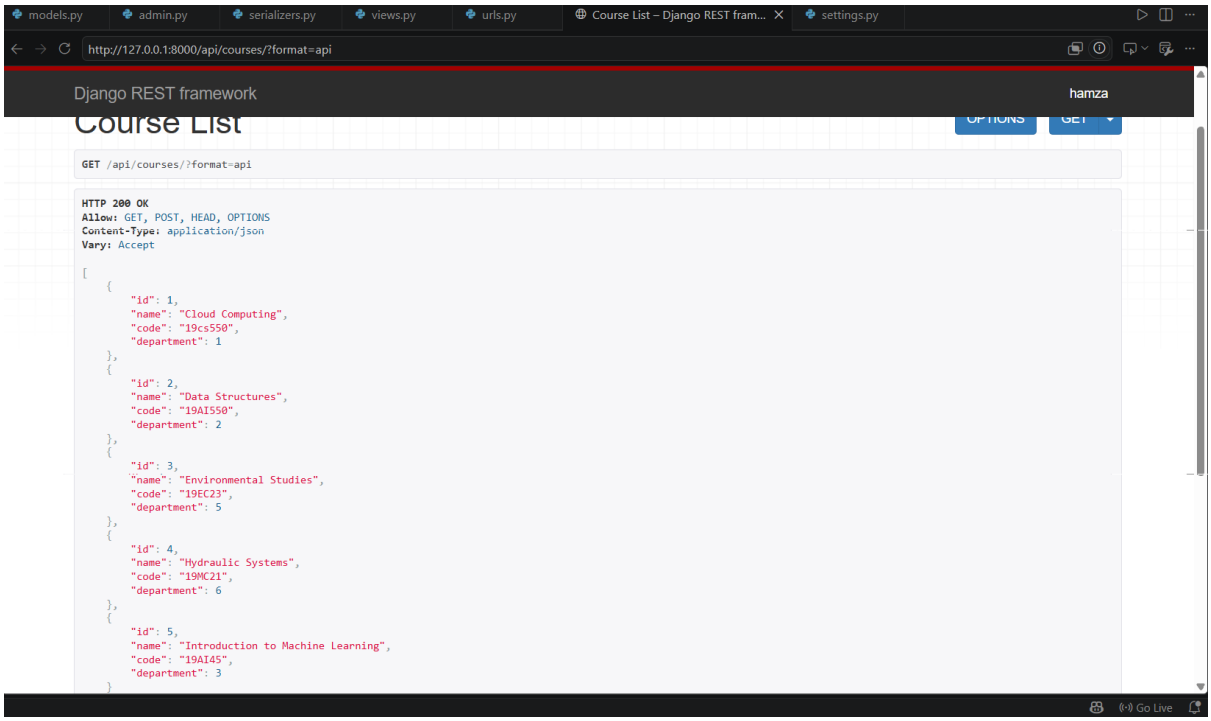
Hint:

- ModelViewSet = ListModelMixin + CreateModelMixin + RetrieveModelMixin + UpdateModelMixin + DestroyModelMixin — all in one class.
- The router generates: GET/POST /courses/ and GET/PUT/PATCH/DELETE /courses/{pk}/ automatically.

Expected Outcome: GET /api/courses/ lists all courses. The custom /students/ action returns enrolled students for a specific course.

OUTCOME:

Courses list:



Student enrolled in a specific course: